

PROJECT PROPOSAL

EECS 4314 - Group 11

May 29, 2026

Name	Email
Karine Davtyan	karineda@my.yorku.ca
Angelina De Santis	adsantis@my.yorku.ca
Yawei Dong	ydong77@my.yorku.ca
Manoj Ravichandran	manojr17@my.yorku.ca
Samin Sharif	samin23@my.yorku.ca
Owen Zhu	owenz75@my.yorku.ca

Product Name

We chose BookAtlas because our application is like an atlas in how it organizes a user's reading history, current books, future plans, and discussions in one place. The name represents the idea of mapping and understanding your reading patterns, making your literary "world" easy to navigate.

Core Concept & User Functionality

BookAtlas is a dynamic database-driven book tracking platform that allows users to engage with books they have read, are currently reading, or plan to read in the future. Upon registering and logging in, users can build a personal library by adding books and assigning each one a status (read, currently reading, dropped, finished). It also allows users to rate each book out of five stars. Users can organize their books further through a Wishlist and Favourites section, making it easy to track books of personal interest and highlight standout reads. Each user maintains a public profile reflecting their library, ratings, and reading history.

BookAtlas plans to create a community discussion forum designed to bring readers together around the books and genres they care about most. Forums are organized by individual book and by genre, giving users a structured space to dive deep into specific titles or explore broader literary conversations. Within each forum, threads are categorized into 'Spoilers', 'Questions', and 'Theories', ensuring that discussions remain organized, accessible, and respectful of readers at different stages of a book. Users can create threads, post replies, and engage in ongoing conversations with other readers around the books they love.

BookAtlas also has features under consideration for future development, including a followers and following system that would allow users to connect with other readers, as well as a notification system to alert users of forum activity and updates. These features are currently out of scope for this iteration of the project but represent a natural direction for the platform to grow.

Tech Stack

Version Control:

We will use GitHub to manage our codebase. Since it allows us to onboard all our team members quickly and do collaborative work easily. We will follow a mono-repo structure, where the frontend and backend are contained in the same repository.

Frontend:

For the frontend, we will use React because its component-based architecture makes the application easier to maintain and develop efficiently. Another factor in this decision is the team's existing experience with React. Additionally, we will use Shadcn and TailwindCSS to rapidly build and prototype the user interface.

Backend:

For the backend, we are currently undecided between FastAPI or Express.js, since the developers on our team have different experiences. We are leaning towards FastAPI for its rapid API development and built-in validation, while Express.js remains a strong option due to its easy integration with React and flexibility. We plan to solidify our decision in the project design document after we receive feedback on our proposal.

Database:

For the database, we are also undecided between MongoDB (NoSQL, unstructured data) and Supabase (PostgreSQL, Backend-as-a-Service). In the most recent lecture, Professor Uddin also mentioned the use of data lakehouses (e.g., Azure Databricks), which we plan to seek more feedback on through our project proposal and by speaking with him during the next class.

Deployment:

For deployment, we plan on using either Google Cloud Run or Render. We are leaning towards Render because it can deploy directly from the GitHub repository without a container or Dockerfile, making it much simpler to use. It also supports setting up a free account without entering billing information. If using Cloud Run, we will use Docker to containerize the services before deployment.

Project Requirements | System Architecture | Learning Goals

Our backend will be separated into four coarse-grained services: Auth Service, Book Service, Book Library / List Service, and Discussion Service. All services will share a monolithic database, and the frontend will be deployed separately, which is all consistent with the service-based architecture style discussed in class. This architecture also satisfies the project requirement of building a dynamic web-based application that maintains state using a database.

Our service-based architecture will include the following services:

Auth Service (User System) Responsibilities:

- Handles all authentication — registration, login, logout, session management
- Stores user profile information — username, email, bio, profile picture, user metadata
- Manages password hashing and credential verification
- Tracks user preferences and account settings

Book Service (Global Catalog System) Responsibilities:

- Maintains the central book catalog for the platform — title, author, genre, cover image, description, ISBN.
- Handles adding, updating, and removing books from the global platform catalog.
- Manages Book Search functionality, including potential external API scraping to fetch book covers and metadata for a complete catalog experience.
- Stores and aggregates global book metrics, such as computing the average 5-star rating based on individual user submissions.

Book Library/List Service (User Book Tracking) Responsibilities:

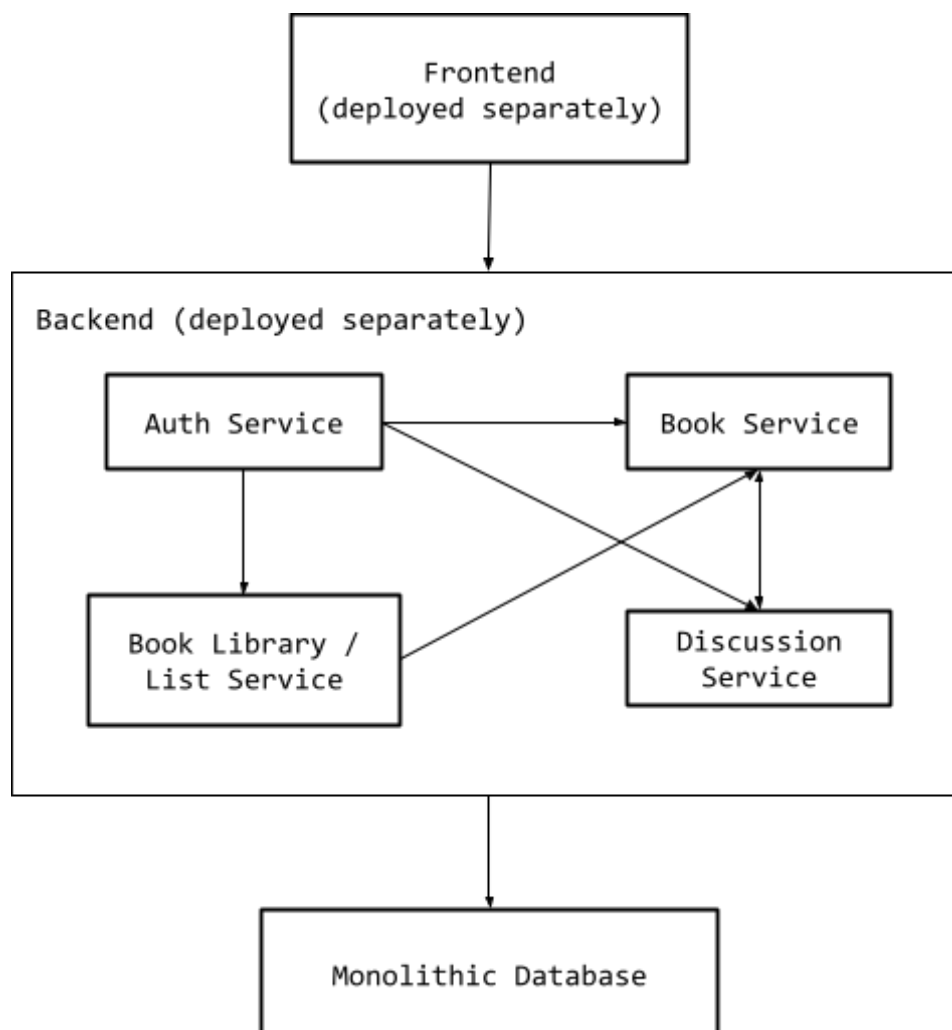
- Manages per-user book statuses (read, currently reading, dropped, finished, want to read) and tracks when a user adds or removes a book from their personal library.
- Manages each user's personal lists — Wishlist, Favourites, and custom collections.
- Handles adding and removing books from each list independently, ensuring a book can appear in multiple lists simultaneously (e.g., Favourites AND Currently Reading).

- Tracks individual reading progress, user-submitted ratings (sent to the Book Service for global aggregation), and status changes over time.
- Stores list history so users can easily view their previously read or dropped books.

Discussion Service (Forum System) Responsibilities:

- Creates and manages discussion forums scoped to each book and each genre
- Handles thread creation, editing, and deletion within each forum
- Categorizes threads by type — Spoilers, Questions, Theories
- Manages posts and replies within threads — hierarchical nested structure
- Tracks which user authored each thread and post
- Handles flagging or moderating forum content
- Supports searching threads by keyword, category, or book

This is a preliminary, high-level diagram of our system architecture:



This system design fulfils several course learning goals outlined in the syllabus:

- First, clearly defining our services and how they connect allows us to express them more formally using UML in the project design document. Having distinct services also helps us identify project risks, which we can address better in the detailed project design document. This separation also helps us divide tasks among group members and follow a clear task sequence.
- Furthermore, choosing a service-based architecture and separating the UI from the backend demonstrates our understanding of the differences between software architecture styles.